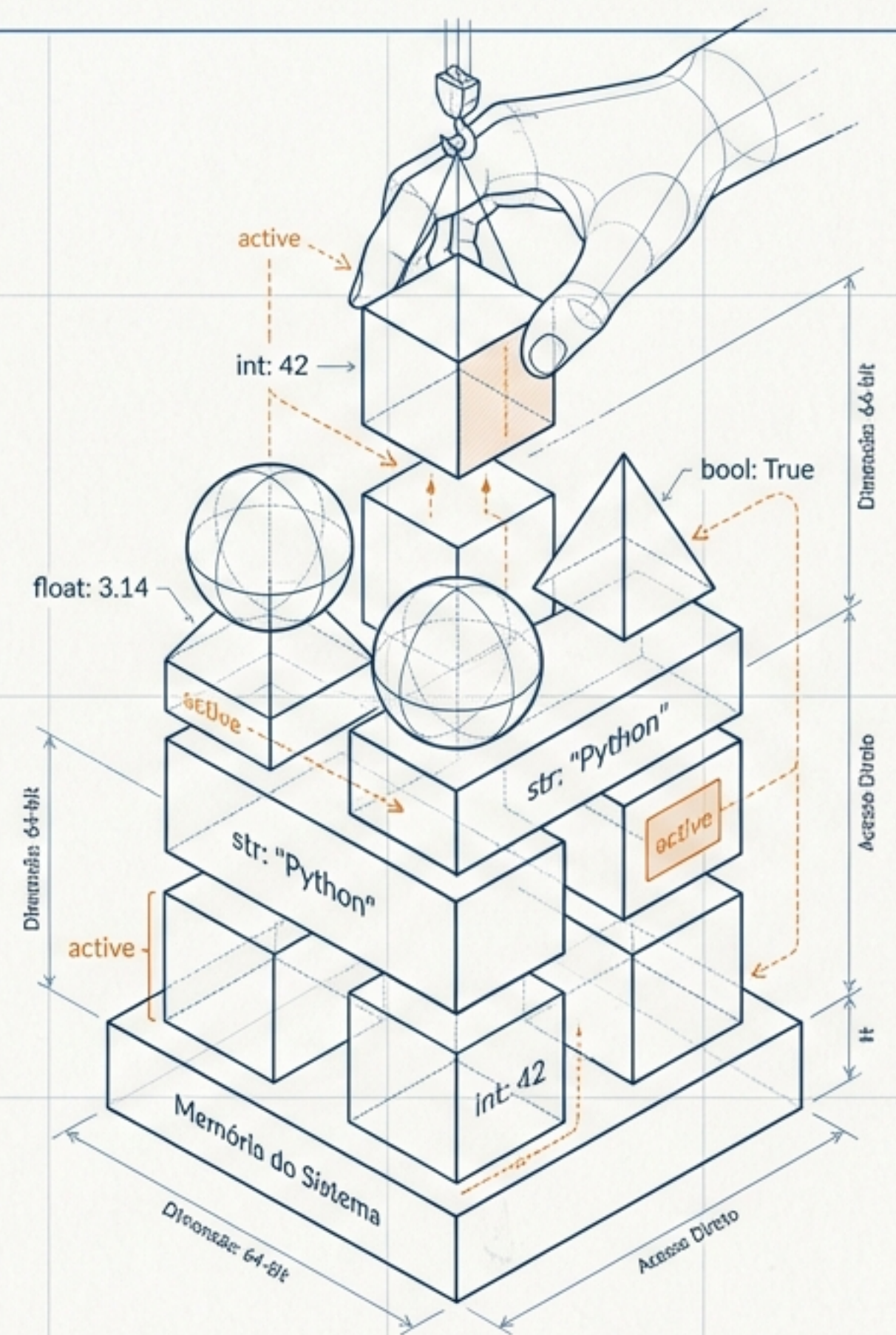


Dominando os Tipos de Dados em Python: A Base da Programação

Uma arquitetura de literais, memória e estruturas lógicas.

JetBrains Mono
Guia de Referência Visual | Python 3.x



A Ilusão do `print()`: Tipo vs. Literal

O **Literal** é a representação fixa no código.
O **Tipo** define como a memória organiza esse valor.
Embora `1` e `1` pareçam iguais para humanos, são entidades distintas para a máquina.

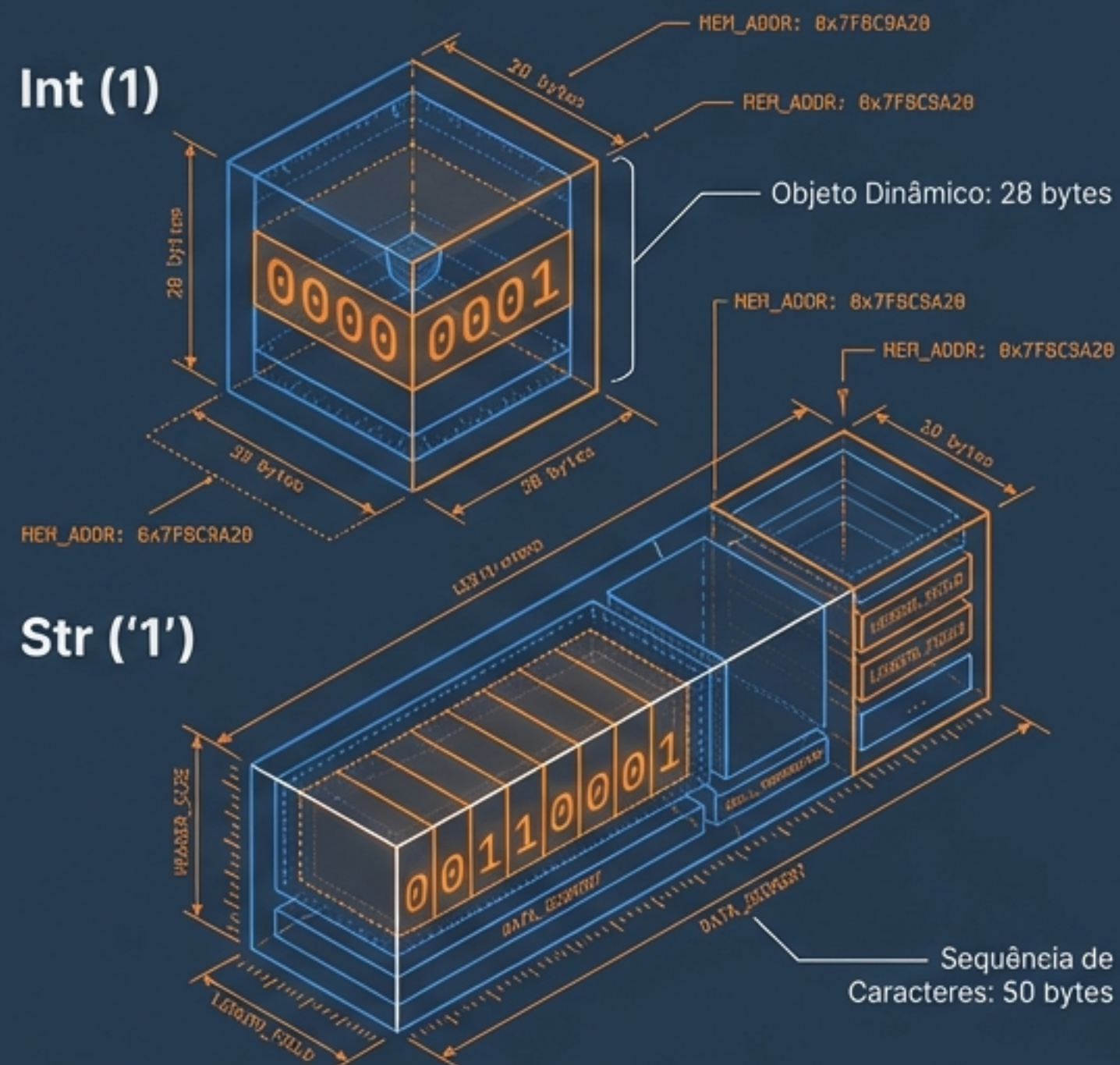
`print(1)` → **1**

`print('1')` → **1**

Note: Visually, they look identical here.

Int (1)

Str ('1')



Technical Blueprint meets Modern Editorial

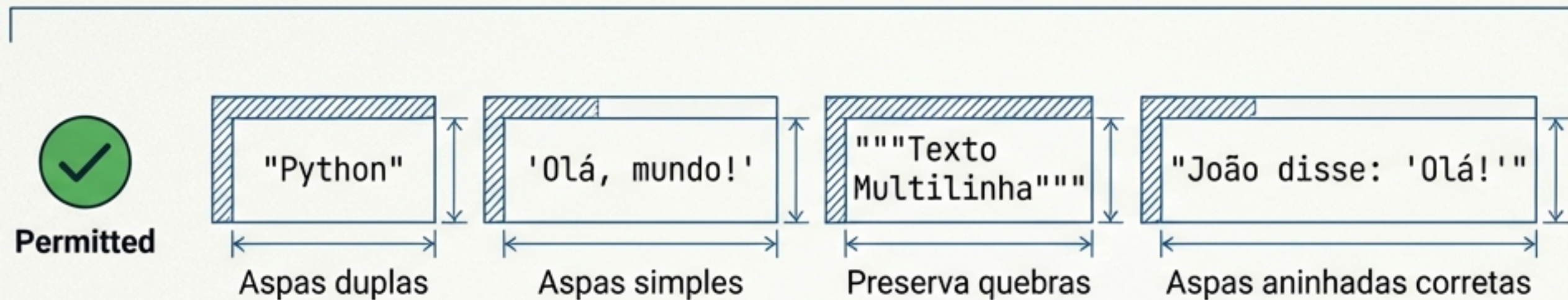
```
import sys
print(sys.getsizeof(1))    # Saída: 28
print(sys.getsizeof('1')) # Saída: 50
```

A função **print()** formata para legibilidade humana, mascarando a complexidade interna da alocação de memória.



Strings: A Camada de Comunicação

Sequência de caracteres delimitada por aspas.

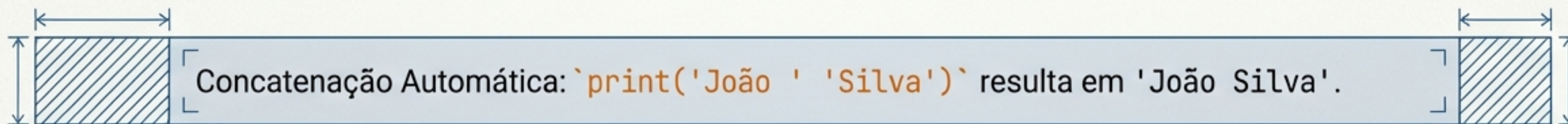
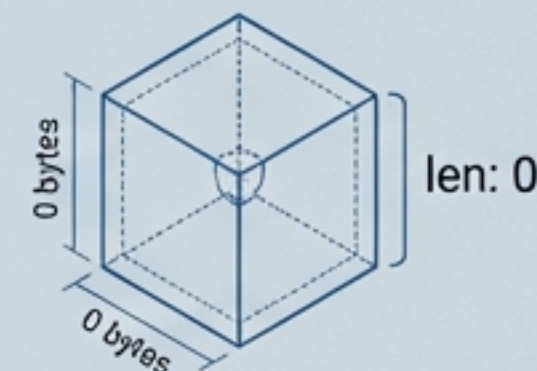


O Conceito de Vazio

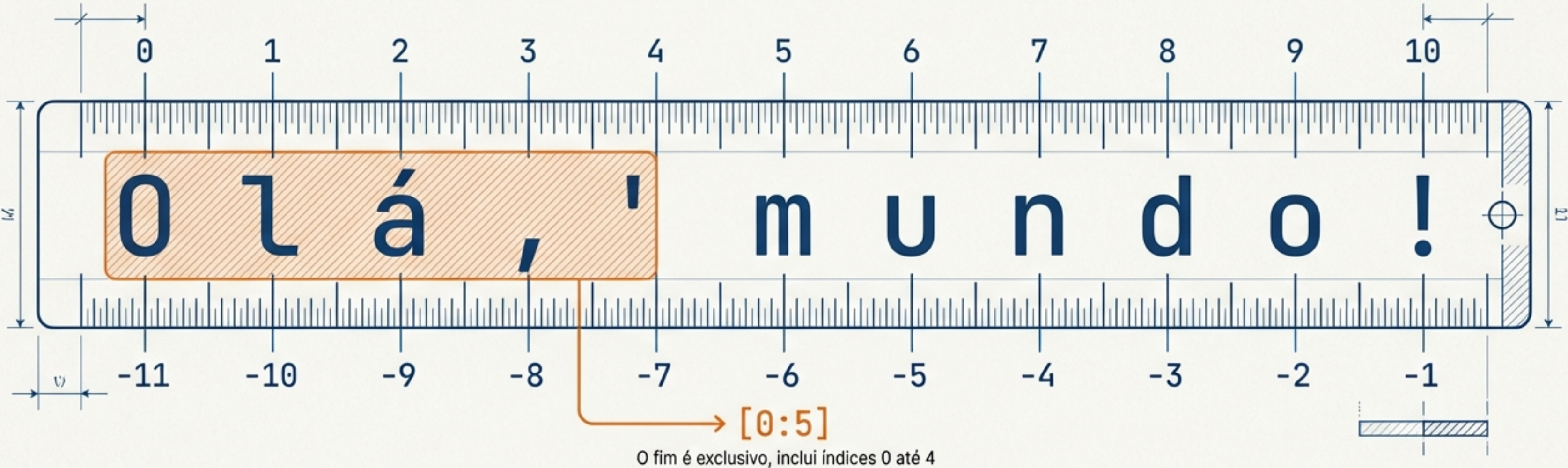
```
string_vazia = ''
```

Uma string vazia é uma sequência válida de ZERO caracteres.

```
len('') # Retorna 0
```

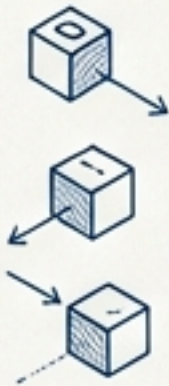


Manipulando Texto: Indexação e Slicing



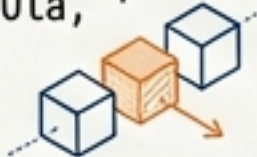
Acesso Direto

`mensagem[0] → 'O'`
`mensagem[-1] → '!'`
`mensagem[5] → 'm'`
Acesso por índice positivo ou negativo.



Slicing

`mensagem[0:5] → 'Olá, '`
(O fim é exclusivo, intervalo [0, 5)).
`mensagem[5:] → ', mundo!'`
`mensagem[:5] → 'Olá, '`



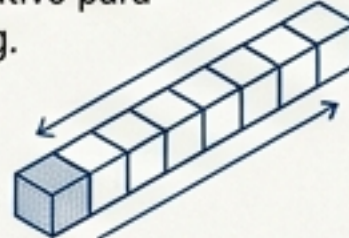
Passo/Step

`mensagem[0:11:2] → 'Oámno'`
Pula 2 caracteres a cada passo.



Inversão

`mensagem[::-1] → '!odnum ,álO'`
Usa passo negativo para reverter a string.



Erro: Acessar índice `[12]` gera `IndexError`. Sempre verifique com `len()`.

Números Inteiros (int): Contagem Precisa

Números sem fração. Tamanho dinâmico (min 28 bytes).



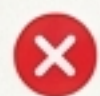
Valid Syntax

11_111_111

← Underscore como
separador visual

-5

+10



Invalid Syntax

11 111 111

← ✗ Erro de Espaços

11,111,111

← ✗ Vírgula cria Tupla, não Int

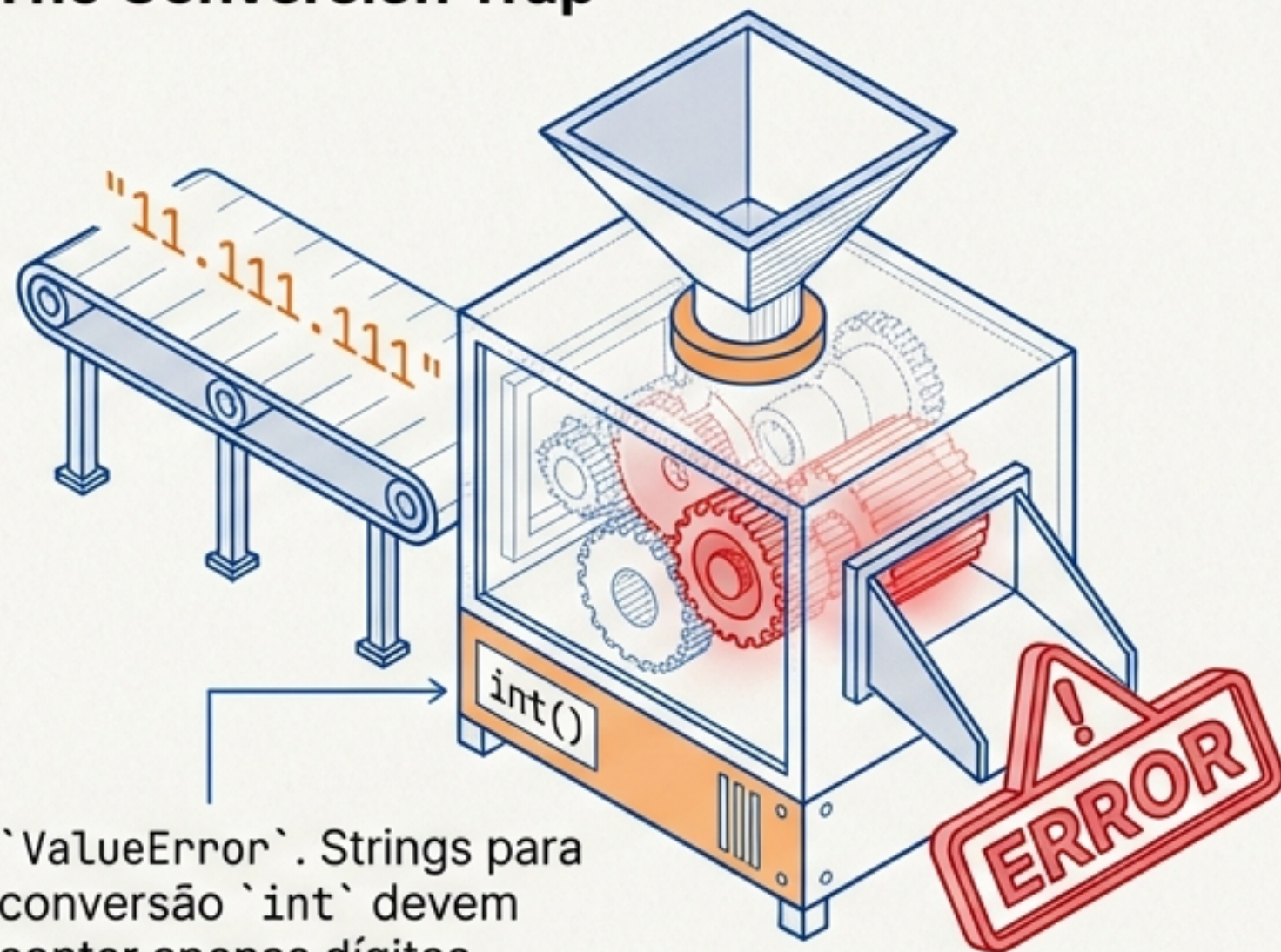


Logic Error

11.111.111

← ⚠ Ponto cria Float ou Erro

The Conversion Trap



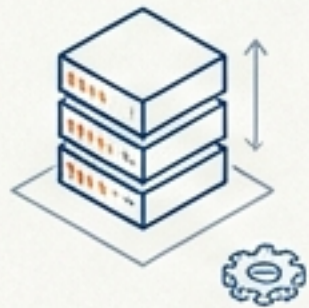
Dica: Python 3.6+ permite underscores ('_') para legibilidade humana.

Ponto Flutuante (**float**): A Arte da Medida



Separador Decimal
Obrigatório

25



O Perigo da Vírgula

❌ Erro Comum

```
print(2,5)
```

2 5

Resultado: `2 5`
Vírgula separa argumentos.

✅ Forma Correta

```
print(2.5)
```

2.5

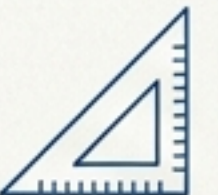
Resultado: `2.5`
Ponto define float.

Regras de Sintaxe

Omissão de Zero

.4 == 0.4

4. == 4.0



Padrão IEEE 754. Ocupam 24 bytes.

Notação Científica e Números Complexos

Notação Científica: A Escala

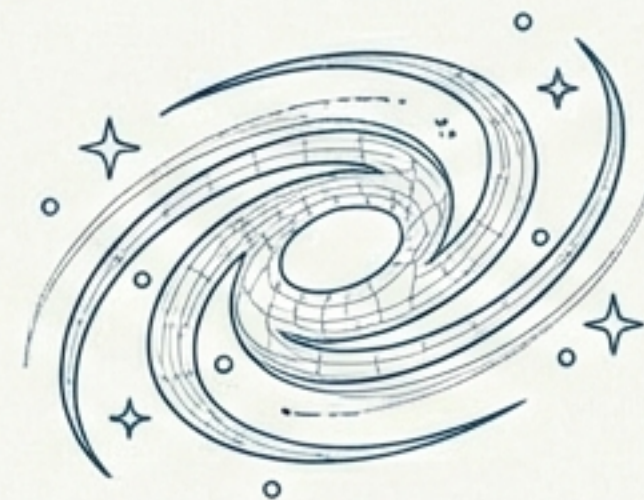


0.00...01
1e-22



Base **e** Expoente

Base * 10^{Expoente}

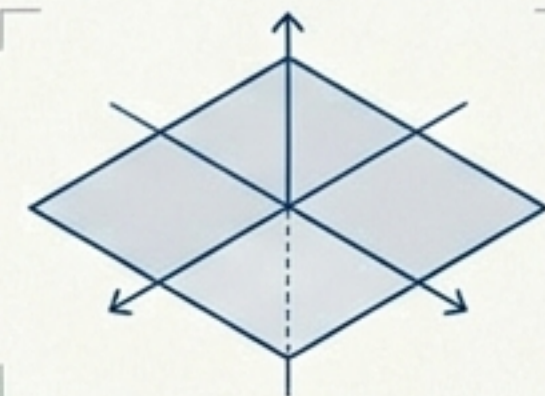


Constante de Planck
6.62607e-34

Números Complexos

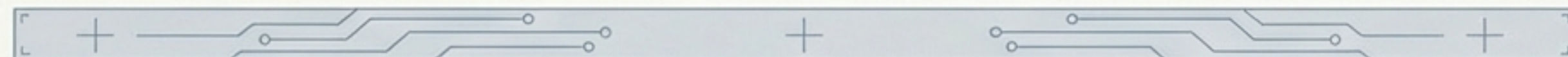
Formato: a + bj
(onde **j** é imaginário).

```
numero = 2 + 3j  
print(type(numero))  
# Saída: <class 'complex'>
```



O Python escolhe automaticamente a notação mais econômica (decimal vs. científica) para exibição.

Booleans (**bool**): A Lógica Binária

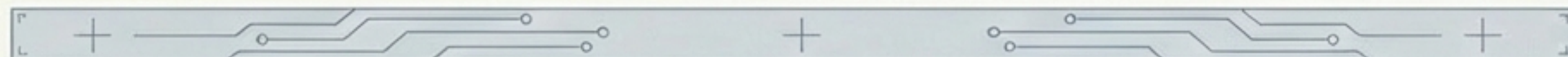


Booleans são subtipos de Inteiros. A matemática funciona:

```
print(True + True)      # Saída: 2
print(True + False)     # Saída: 1
print(False * 10)        # Saída: 0
```



Aplicando a Lógica: Controle de Fluxo



Negação.

Estado inicial comum.

```
`if not condicao:`  
executa se for falso.
```

Confirmação.

Funções retornam **True** para sucesso.

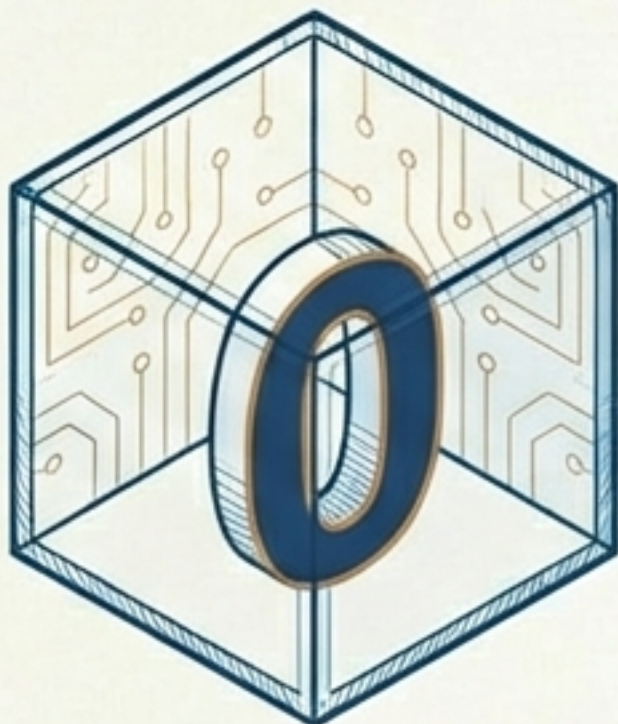
```
`if True:`  
executa sempre.
```



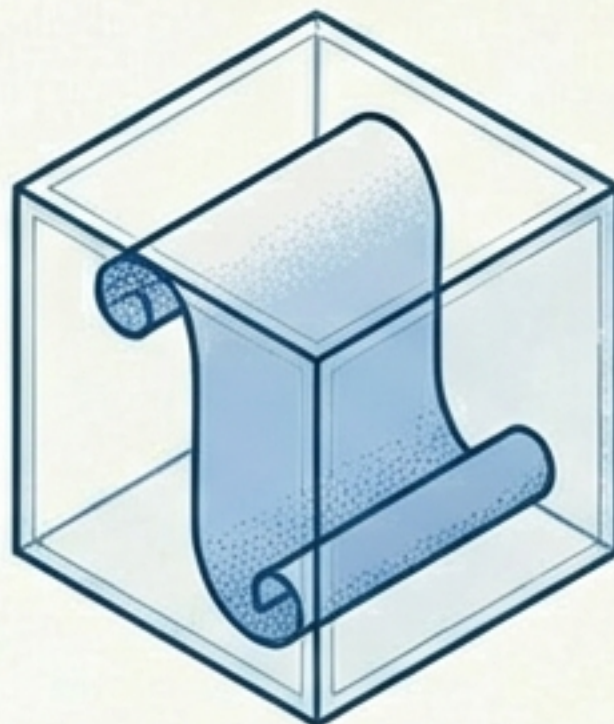
"Você nunca receberá 'Talvez' como resposta do Python."

NoneType: A Ausência Significativa

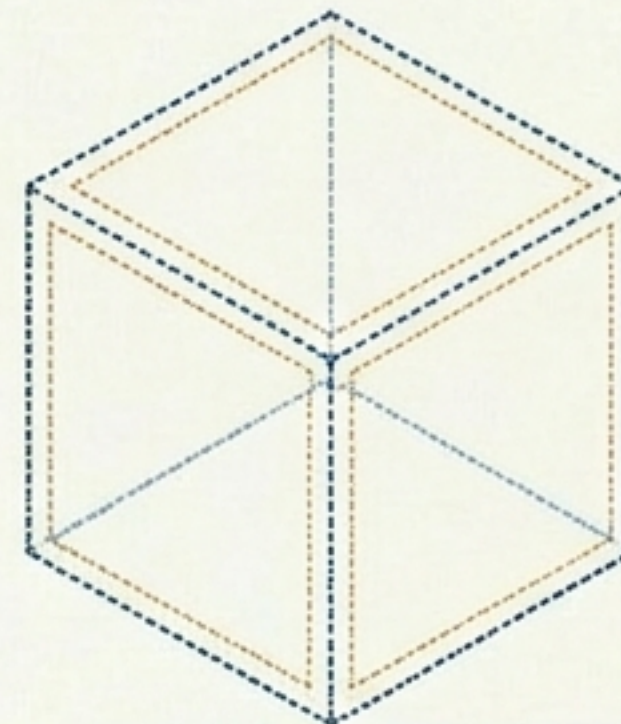
The Three Boxes (Visual Metaphor)



Inteiro (0)



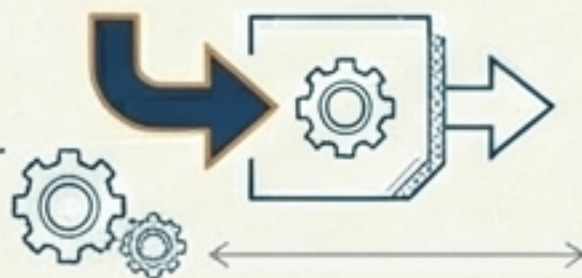
String Vazia ("")



'None' (Ausência)

'None' não é zero. Não é string vazia. É um placeholder para 'sem valor'.

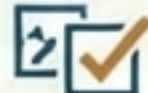
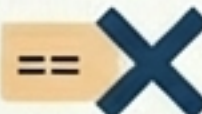
Technical Details



Implicit Return

Funções sem `return`
devolvem `None`
automaticamente.

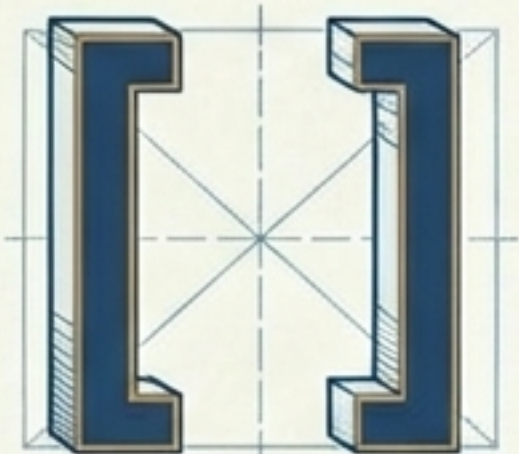
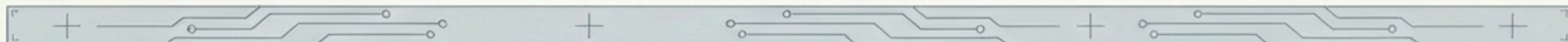
Identity Check

if variavel is None:  or 

Use sempre `if variavel is None:`
(Identidade) e não `==`.

O Futuro: Estruturas Compostas

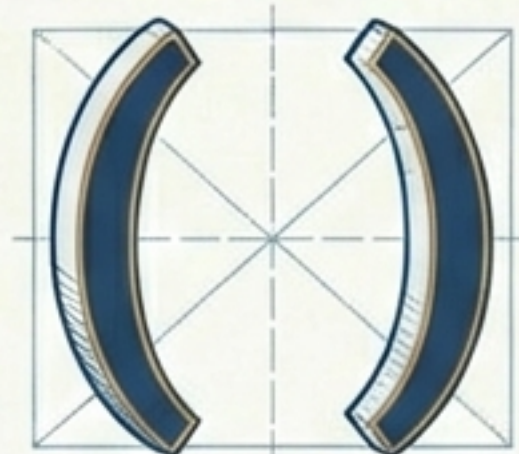
Como átomos formam moléculas, tipos primitivos formam estruturas. *A sintaxe define o container.*



Lista

Ordenada, Mutável.

[1, 2]



Tupla

Ordenada, Imutável.

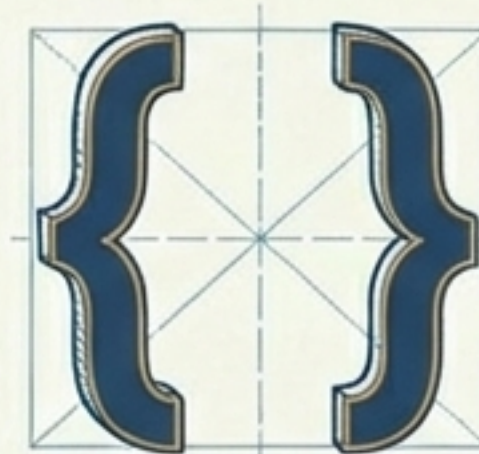
(1, 2)



Dicionário

Chave-Valor.

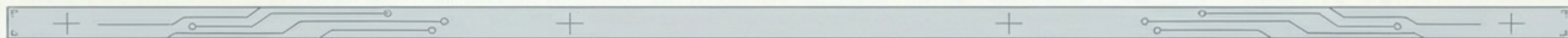
{'a':1}



Conjunto

Único, Não Ordenado.

{1, 2}



Atenção: `{ }` define Dicionários e Conjuntos.
A diferença é o par `chave:valor`.



O Blueprint dos Dados (Resumo)

Text (str)	Aspas <code>' ' " "</code>	Imutável	Sequência
Numbers	<code>int</code> (Inteiros, <code>_</code>), <code>float</code> (Decimais, <code>.</code>), <code>complex</code> (<code>j</code>)		
Logic (bool)	<code>True</code> (<code>1</code>), <code>False</code> (<code>0</code>)		Álgebra booleana
Void (None)	Ausência de valor		Placeholder

Escolher o tipo de dado correto é o primeiro passo para um código eficiente e livre de erros.